mc_control_epsilon

October 12, 2026

[10]:
```python
import os, sys;
sys.path.append(os.path.abspath('..')) # for importing the parent dirs
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict
from common.gridworld import GridWorld
# from common.utils import greedy_probs


def greedy_probs(Q, state, epsilon=0, action_size=4):
    qs = [Q[(state, action)] for action in range(action_size)]
    max_action = np.argmax(qs)

    base_prob = epsilon / action_size
    action_probs = {action: base_prob for action in range(action_size)}  #{0: /
 ↪4, 1: /4, 2: /4, 3: /4}
    action_probs[max_action] += (1 - epsilon)
    return action_probs


class McAgent:
    def __init__(self, epsilon=0.1, alpha=0.1):
        self.gamma = 0.9
        self.epsilon = epsilon  # (     ) -
        self.alpha = alpha    # (     ) Q
        self.action_size = 4

        random_actions = {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25}
        self.pi = defaultdict(lambda: random_actions)
        self.Q = defaultdict(lambda: 0)
        # self.cnts = defaultdict(lambda: 0)
        self.memory = []

    def get_action(self, state):
        action_probs = self.pi[state]
        actions = list(action_probs.keys())
        probs = list(action_probs.values())
```

```
        return np.random.choice(actions, p=probs)

    def add(self, state, action, reward):
        data = (state, action, reward)
        self.memory.append(data)

    def reset(self):
        self.memory.clear()

    def update(self):
        G = 0.0
        total_q_delta = 0.0
        greedy_changes = 0
        for data in reversed(self.memory):
            state, action, reward = data
            G = self.gamma * G + reward
            key = (state, action)
            # Monte Carlo Control:    Q
            old_q = self.Q[key]
            self.Q[key] += (G - old_q) * self.alpha
            total_q_delta += abs(self.Q[key] - old_q)

            #
            old_greedy = max(self.pi[state], key=self.pi[state].get)
            self.pi[state] = greedy_probs(self.Q, state, self.epsilon)
            new_greedy = max(self.pi[state], key=self.pi[state].get)
            if old_greedy != new_greedy:
                greedy_changes += 1

        #    Return       G         G0
        mean_q_delta = total_q_delta / len(self.memory) if self.memory else 0.0
        return G, mean_q_delta, greedy_changes



def moving_average(values, window=200):
    #   Episode         window
    values = np.asarray(values, dtype=float)
    if len(values) < window:
        window = len(values)
    if window == 0:
        return np.array([]), np.array([])
    averaged = np.convolve(values, np.ones(window) / window, mode='valid')
    indices = np.arange(window, len(values) + 1)
    return indices, averaged
```

```
def plot_learning_curves(histories, parameter_name, parameter_values,␣
 ↪window=200):
    fig, axes = plt.subplots(2, 1, figsize=(11, 8), sharex=True)
    for value in parameter_values:
        hist = histories[value]
        x, return_avg = moving_average(hist['returns'], window)
        _, delta_avg = moving_average(hist['q_delta'], window)
        axes[0].plot(x, return_avg, label=f'{parameter_name}={value}')
        axes[1].plot(x, delta_avg, label=f'{parameter_name}={value}')

    axes[0].set_title('Episode Return (discounted G0)')
    axes[0].set_ylabel('Moving average of G0')
    axes[1].set_title('Mean absolute Q update per episode')
    axes[1].set_ylabel('Moving average of mean |dQ|')
    axes[1].set_xlabel('Episode')
    for ax in axes:
        ax.grid(alpha=0.3)
        ax.legend()
    fig.tight_layout()
    plt.show()


def summarize_q_change(snapshot, final_q):
    #          State-Action      0
    keys = set(snapshot) | set(final_q)
    if not keys:
        return 0.0, 0
    mean_change = np.mean([abs(final_q.get(k, 0.0) - snapshot.get(k, 0.0))
                           for k in keys])
    return float(mean_change), len(keys)

epsilon_values = [0.01, 0.05, 0.1, 0.3, 0.5]
episodes = 10000


def run_epsilon(epsilon, episodes=10000):
    env = GridWorld()
    agent = McAgent(epsilon=epsilon, alpha=0.1)
    history = {'returns': [], 'q_delta': [], 'greedy_changes': []}
    snapshot_at = min(1000, episodes)
    q_snapshot = {}

    print(f'epsilon = {epsilon}    ')
    for episode in range(episodes):
        state = env.reset()
        agent.reset()
```

```
        #          MC
        while True:
            action = agent.get_action(state)
            next_state, reward, done = env.step(action)
            agent.add(state, action, reward)

            if done:
                G0, mean_q_delta, changes = agent.update()
                break
            state = next_state

        history['returns'].append(G0)
        history['q_delta'].append(mean_q_delta)
        history['greedy_changes'].append(changes)

        if episode + 1 == snapshot_at:
            q_snapshot = dict(agent.Q)

        if (episode + 1) % 1000 == 0 or episode + 1 == episodes:
            last = min(1000, episode + 1)
            print(f'Episode {episode + 1:5d}/{episodes}: '
                  f'  {last}    G0={np.mean(history["returns"][-last:]):.3f}, '
                  f'  |dQ|={np.mean(history["q_delta"][-last:]):.5f}, '
                  f'Greedy Action  ={sum(history["greedy_changes"][-last:])} ')

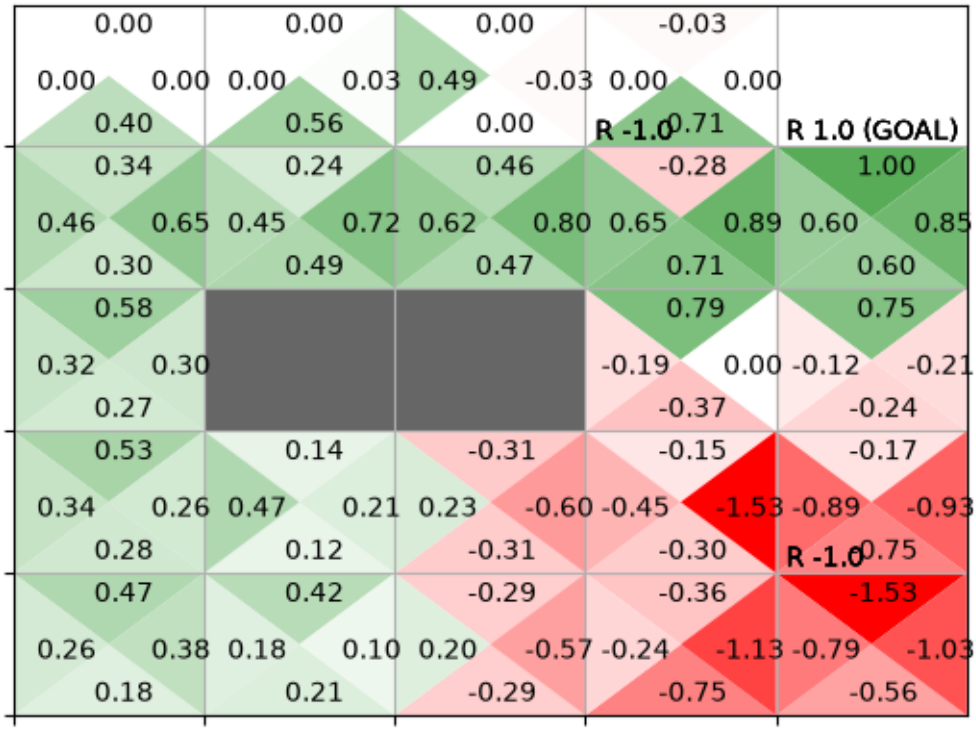
    avg_difference, n_pairs = summarize_q_change(q_snapshot, agent.Q)
    print(f'epsilon = {epsilon}     ')
    print(f'Q   ({snapshot_at} -> {episodes} Episode): '
          f'      ={avg_difference:.5f}, State-Action  ={n_pairs} ')
    env.render_q(agent.Q)  #     Q   Greedy Policy
    return agent, history

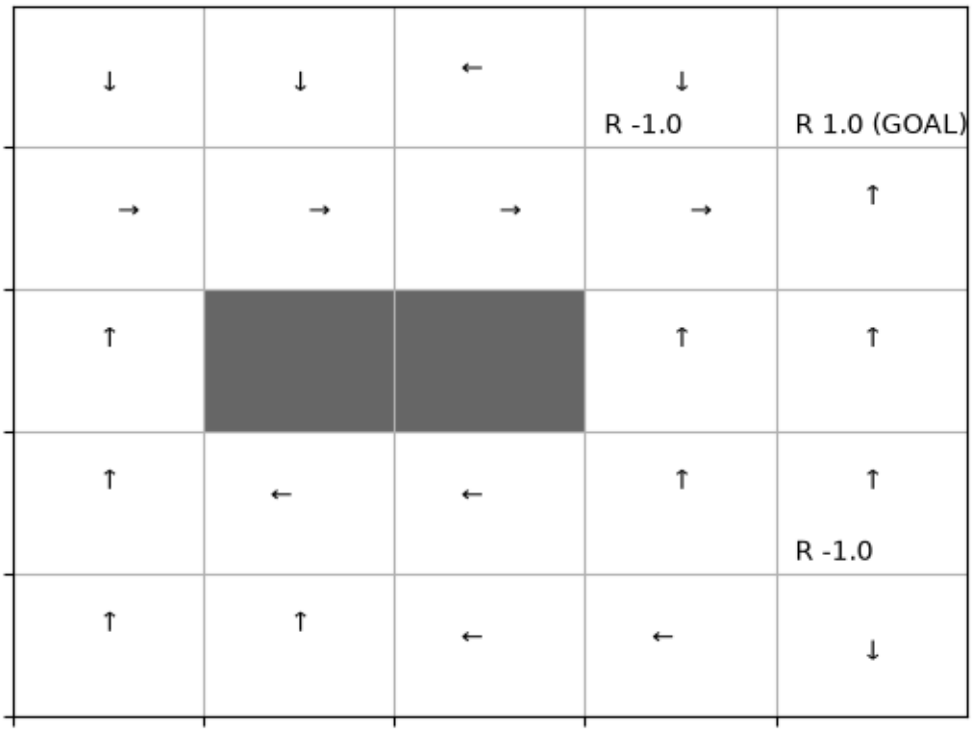

results = {}
history_results = {}
for epsilon in epsilon_values:
    results[epsilon], history_results[epsilon] = run_epsilon(epsilon, episodes)

# epsilon       : Return   Q
plot_learning_curves(history_results, 'epsilon', epsilon_values, window=200)
```

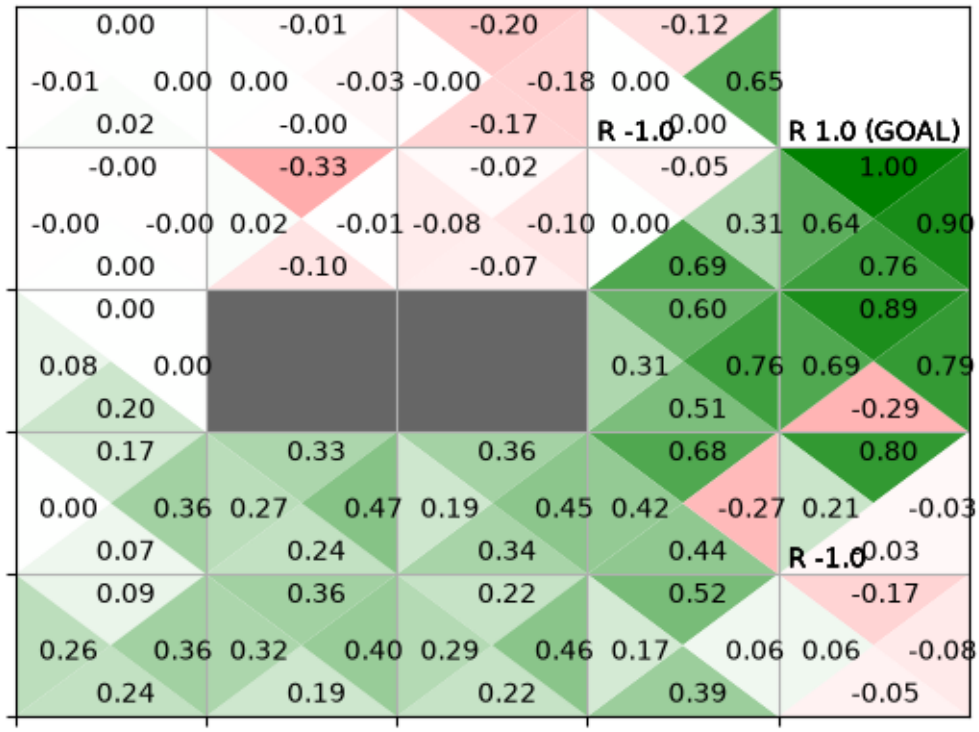
```
epsilon = 0.01
Episode  1000/10000:   1000    G0=0.472,   |dQ|=0.00140, Greedy Action  =51
Episode  2000/10000:   1000    G0=0.474,   |dQ|=0.00077, Greedy Action  =0
Episode  3000/10000:   1000    G0=0.472,   |dQ|=0.00100, Greedy Action  =0
Episode  4000/10000:   1000    G0=0.473,   |dQ|=0.00082, Greedy Action  =0
Episode  5000/10000:   1000    G0=0.472,   |dQ|=0.00100, Greedy Action  =0
Episode  6000/10000:   1000    G0=0.472,   |dQ|=0.00080, Greedy Action  =0
```

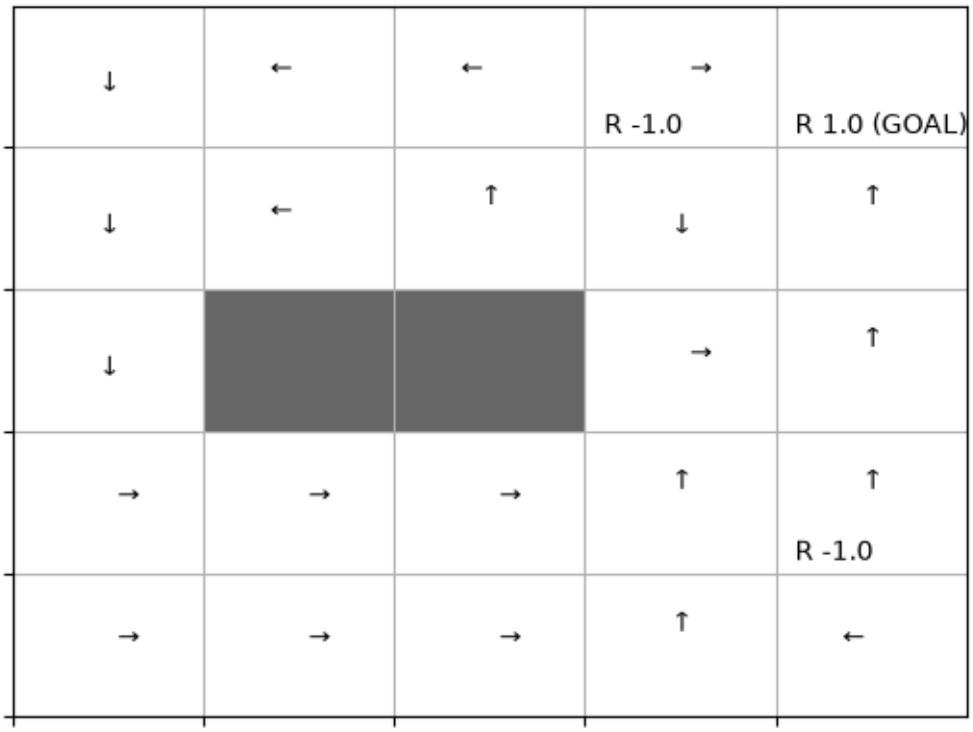
```
Episode  7000/10000:    1000    G0=0.408,    |dQ|=0.00130, Greedy Action
 =150
Episode  8000/10000:    1000    G0=0.382,    |dQ|=0.00100, Greedy Action  =0
Episode  9000/10000:    1000    G0=0.383,    |dQ|=0.00080, Greedy Action  =0
Episode 10000/10000:    1000    G0=0.412,    |dQ|=0.00105, Greedy Action  =1
epsilon = 0.01
Q    (1000 -> 10000 Episode):       =0.17224, State-Action  =88
```



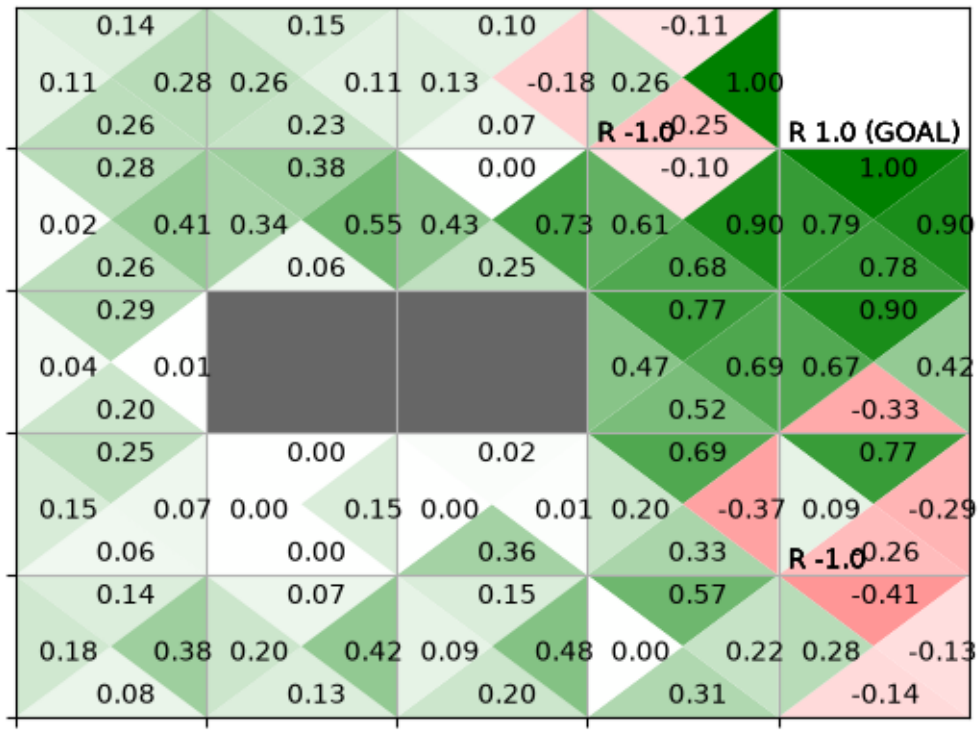


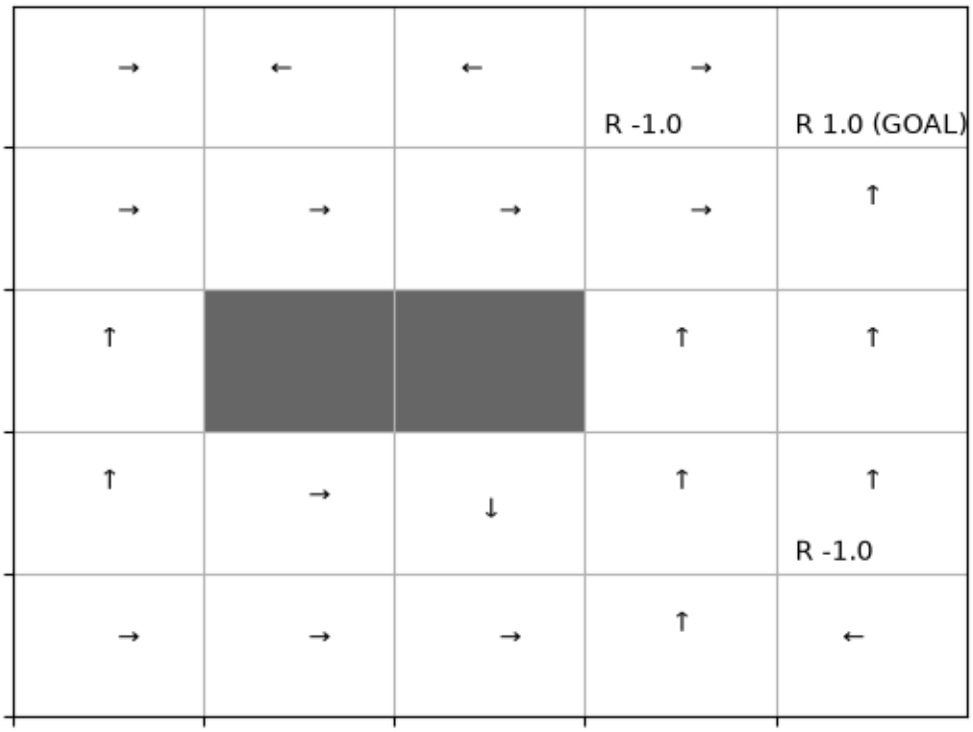

```
epsilon = 0.05
Episode  1000/10000:   1000    G0=0.435,   |dQ|=0.00560, Greedy Action
 =106
Episode  2000/10000:   1000    G0=0.433,   |dQ|=0.00474, Greedy Action  =30
Episode  3000/10000:   1000    G0=0.439,   |dQ|=0.00463, Greedy Action
 =309
Episode  4000/10000:   1000    G0=0.432,   |dQ|=0.00549, Greedy Action  =45
Episode  5000/10000:   1000    G0=0.433,   |dQ|=0.00457, Greedy Action  =25
Episode  6000/10000:   1000    G0=0.406,   |dQ|=0.00494, Greedy Action  =34
Episode  7000/10000:   1000    G0=0.439,   |dQ|=0.00441, Greedy Action  =17
Episode  8000/10000:   1000    G0=0.436,   |dQ|=0.00460, Greedy Action  =78
Episode  9000/10000:   1000    G0=0.440,   |dQ|=0.00458, Greedy Action  =37
Episode 10000/10000:   1000    G0=0.435,   |dQ|=0.00525, Greedy Action  =11
epsilon = 0.05
Q   (1000 -> 10000 Episode):       =0.09461, State-Action  =88
```


0.00 -0.01 -0.20 -0.12
-0.01 0.00 0.00 -0.03 -0.00 -0.18 0.00 0.65
0.02 -0.00 -0.17 R -1.0 0.00 R 1.0 (GOAL)
-0.00 -0.33 -0.02 -0.05 1.00
-0.00 -0.00 0.02 -0.01 -0.08 -0.10 0.00 0.31 0.64 0.90
0.00 -0.10 -0.07 0.69 0.76
0.00 0.60 0.89
0.08 0.00 0.31 0.76 0.69 0.79
0.20 0.51 -0.29
0.17 0.33 0.36 0.68 0.80
0.00 0.36 0.27 0.47 0.19 0.45 0.42 -0.27 0.21 -0.03
0.07 0.24 0.34 0.44 R -1.0 0.03
0.09 0.36 0.22 0.52 -0.17
0.26 0.36 0.32 0.40 0.29 0.46 0.17 0.06 0.06 -0.08
0.24 0.19 0.22 0.39 -0.05

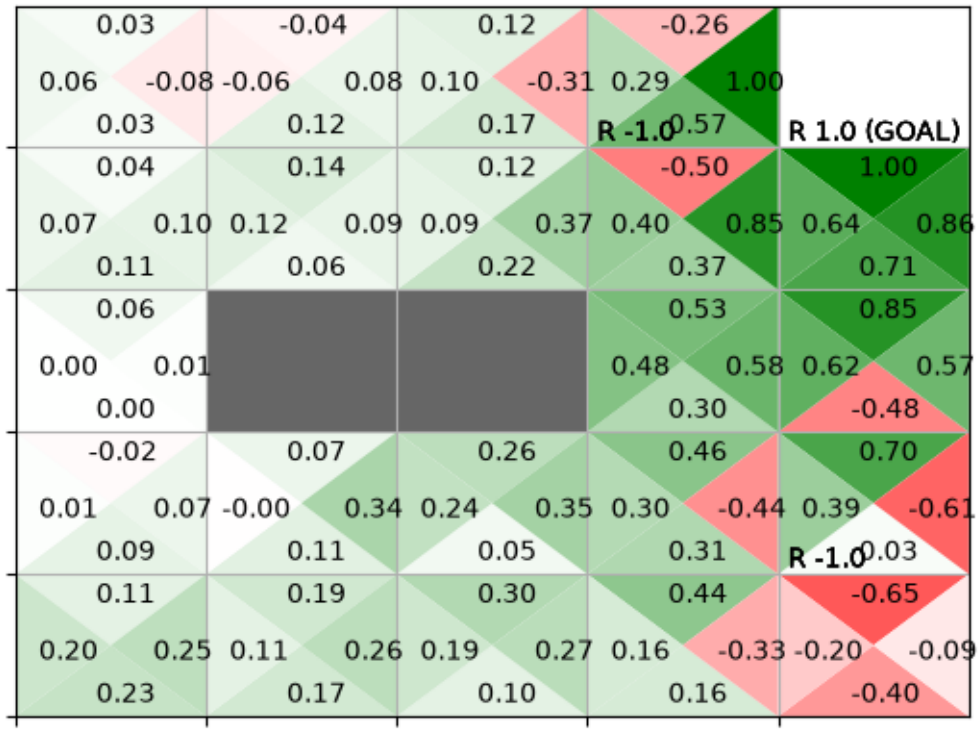

↓ ← ← → R -1.0 R 1.0 (GOAL)
↓ ← ↑ ↓ ↑
↓ → ↑
→ → → ↑ ↑ R -1.0
→ → → ↑ ←

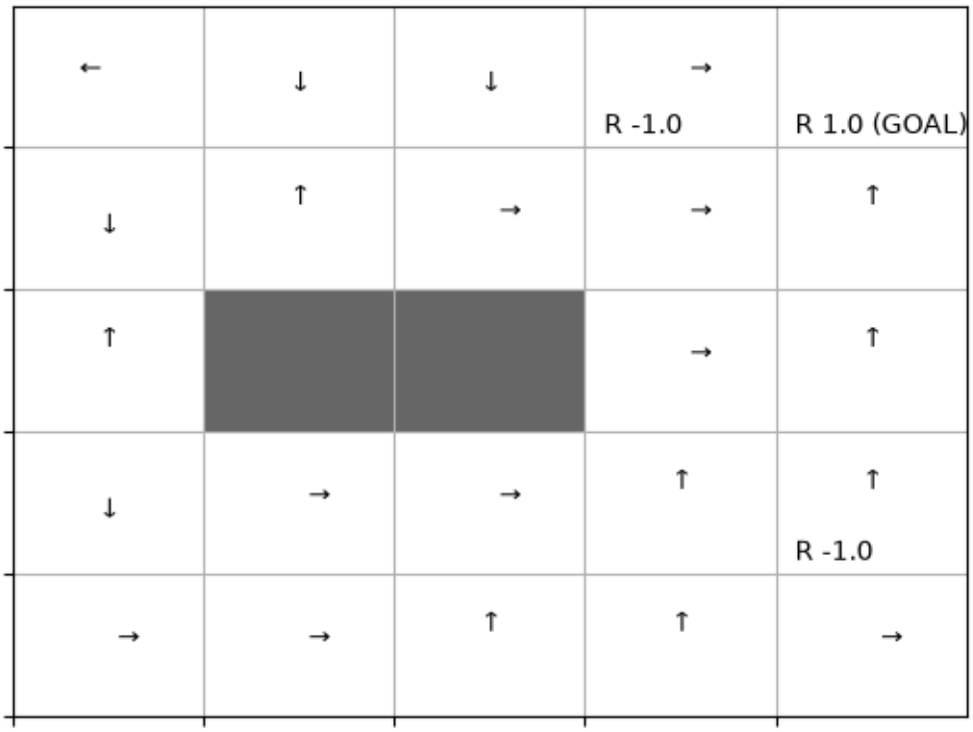
```
epsilon = 0.1
Episode  1000/10000:    1000    G0=0.362,    |dQ|=0.00794, Greedy Action
 =834
Episode  2000/10000:    1000    G0=0.415,    |dQ|=0.00659, Greedy Action  =59
Episode  3000/10000:    1000    G0=0.410,    |dQ|=0.00738, Greedy Action  =55
Episode  4000/10000:    1000    G0=0.408,    |dQ|=0.00731, Greedy Action  =46
Episode  5000/10000:    1000    G0=0.388,    |dQ|=0.00854, Greedy Action
 =116
Episode  6000/10000:    1000    G0=0.413,    |dQ|=0.00731, Greedy Action  =62
Episode  7000/10000:    1000    G0=0.408,    |dQ|=0.00751, Greedy Action  =27
Episode  8000/10000:    1000    G0=0.402,    |dQ|=0.00772, Greedy Action  =94
Episode  9000/10000:    1000    G0=0.402,    |dQ|=0.00697, Greedy Action  =78
Episode 10000/10000:    1000    G0=0.394,    |dQ|=0.00871, Greedy Action
 =150
epsilon = 0.1
Q   (1000 -> 10000 Episode):       =0.16304, State-Action  =88
```



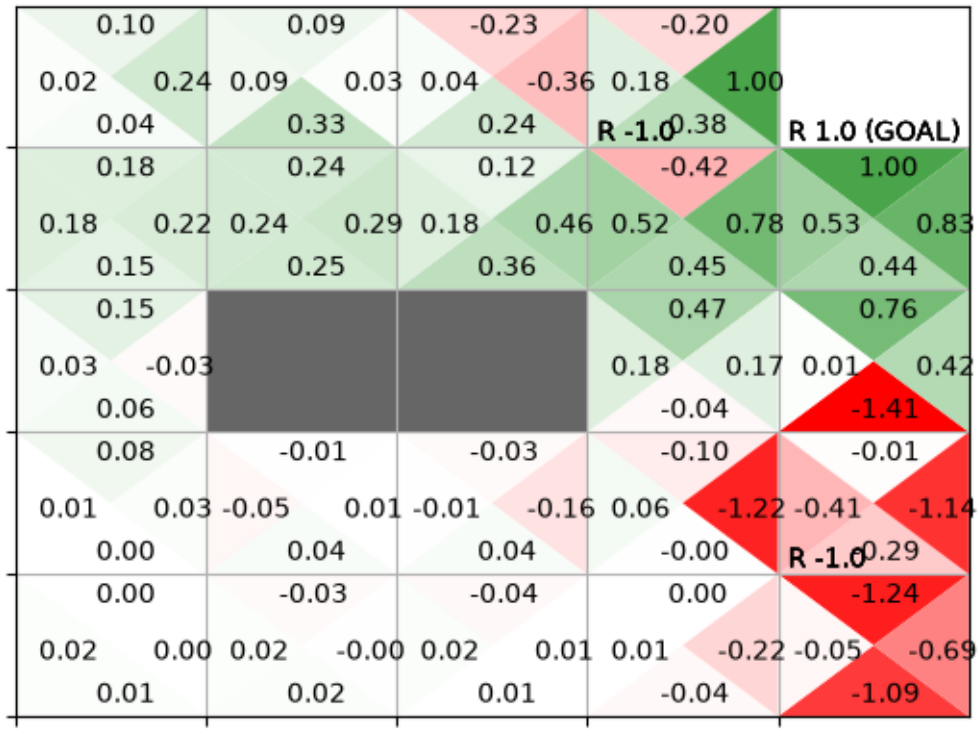


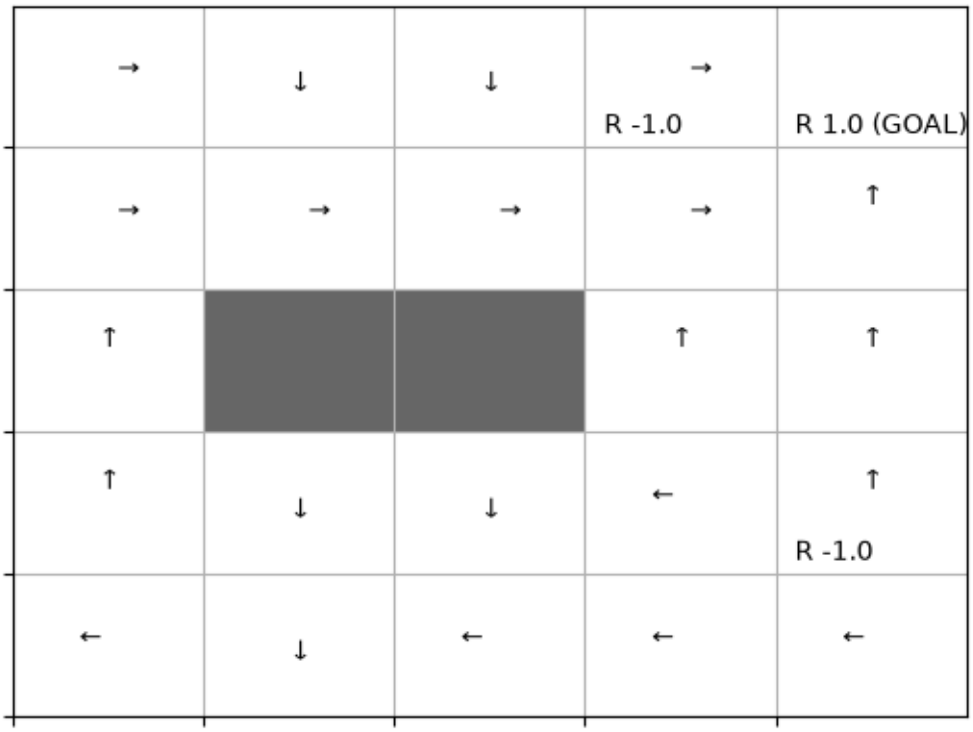

```
epsilon = 0.3
Episode  1000/10000:   1000    G0=0.227,   |dQ|=0.01947, Greedy Action
 =577
Episode  2000/10000:   1000    G0=0.199,   |dQ|=0.02057, Greedy Action
 =665
Episode  3000/10000:   1000    G0=0.229,   |dQ|=0.01779, Greedy Action
 =569
Episode  4000/10000:   1000    G0=0.236,   |dQ|=0.01822, Greedy Action
 =472
Episode  5000/10000:   1000    G0=0.242,   |dQ|=0.01696, Greedy Action
 =545
Episode  6000/10000:   1000    G0=0.218,   |dQ|=0.01843, Greedy Action
 =569
Episode  7000/10000:   1000    G0=0.215,   |dQ|=0.01871, Greedy Action
 =583
Episode  8000/10000:   1000    G0=0.231,   |dQ|=0.01757, Greedy Action
 =484
Episode  9000/10000:   1000    G0=0.246,   |dQ|=0.01783, Greedy Action
 =438
Episode 10000/10000:   1000    G0=0.226,   |dQ|=0.01851, Greedy Action
 =601
epsilon = 0.3
Q   (1000 -> 10000 Episode):       =0.13843, State-Action  =88
```

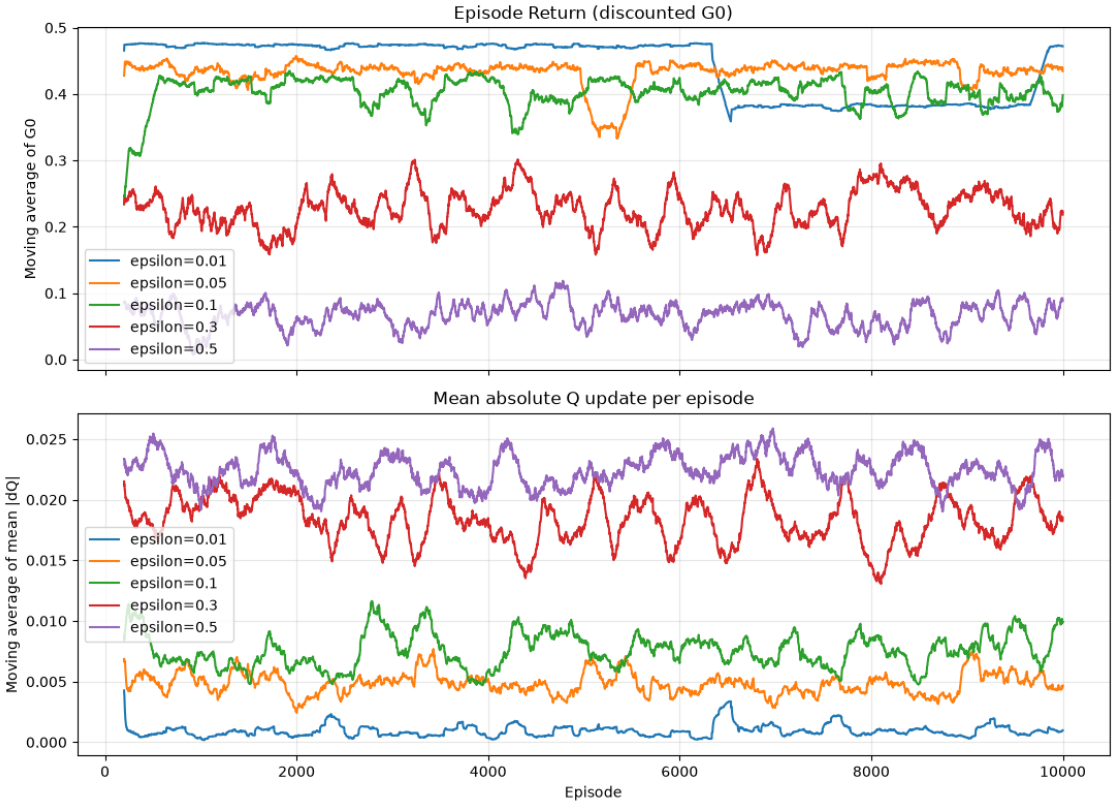

0.03
-0.04
0.12
-0.26
0.06
-0.08
-0.06
0.08
0.10
-0.31
0.29
1.00
0.03
0.12
0.17
R -1.0
0.57
R 1.0 (GOAL)
0.04
0.14
0.12
-0.50
1.00
0.07
0.10
0.12
0.09
0.09
0.37
0.40
0.85
0.64
0.86
0.11
0.06
0.22
0.37
0.71
0.06
0.53
0.85
0.00
0.01
0.48
0.58
0.62
0.57
0.00
0.30
-0.48
-0.02
0.07
0.26
0.46
0.70
0.01
0.07
-0.00
0.34
0.24
0.35
0.30
-0.44
0.39
-0.61
0.09
0.11
0.05
0.31
R -1.0
0.03
0.11
0.19
0.30
0.44
-0.65
0.20
0.25
0.11
0.26
0.19
0.27
0.16
-0.33
-0.20
-0.09
0.23
0.17
0.10
0.16
-0.40


R -1.0
R 1.0 (GOAL)
R -1.0

```
epsilon = 0.5
Episode  1000/10000:    1000    G0=0.061,    |dQ|=0.02252, Greedy Action
 =2077
Episode  2000/10000:    1000    G0=0.068,    |dQ|=0.02271, Greedy Action
 =1810
Episode  3000/10000:    1000    G0=0.063,    |dQ|=0.02177, Greedy Action
 =2062
Episode  4000/10000:    1000    G0=0.064,    |dQ|=0.02161, Greedy Action
 =1999
Episode  5000/10000:    1000    G0=0.083,    |dQ|=0.02190, Greedy Action
 =1714
Episode  6000/10000:    1000    G0=0.069,    |dQ|=0.02313, Greedy Action
 =1936
Episode  7000/10000:    1000    G0=0.077,    |dQ|=0.02333, Greedy Action
 =1772
Episode  8000/10000:    1000    G0=0.051,    |dQ|=0.02262, Greedy Action
 =2328
Episode  9000/10000:    1000    G0=0.059,    |dQ|=0.02251, Greedy Action
 =2126
Episode 10000/10000:    1000    G0=0.076,    |dQ|=0.02213, Greedy Action
 =1797
epsilon = 0.5
Q   (1000 -> 10000 Episode):       =0.13366, State-Action  =88
```




R -1.0
R 1.0 (GOAL)
R -1.0


Episode Return (discounted G0)
Moving average of G0
epsilon=0.01
epsilon=0.05
epsilon=0.1
epsilon=0.3
epsilon=0.5
Mean absolute Q update per episode
Moving average of mean |dQ|
Episode